🎯 Luciteria Collector Cabinet — Setup Guide for Non-Developers

Welcome! This guide will help you run the Collector Cabinet prototype on your computer. Don't worry if you're not technical — we'll walk through every step together.

📋 Table of Contents

🛠 1. Prerequisites — What You Need First

Before you can run the prototype, you need two pieces of software installed on your computer:

What is Node.js and npm?

Node.js is software that lets your computer run JavaScript programs (like our prototype). Think of it like a translator that helps your computer understand the code.

npm (Node Package Manager) comes with Node.js automatically. It's like an app store for code — it downloads all the helper tools our prototype needs to work.

✅ Check If You Already Have Them

Before installing anything, let's check if you already have Node.js and npm installed.

On Mac:

1. **Open Terminal:**
   - Press `Command (⌘) + Space` to open Spotlight
   - Type "Terminal" and press Enter
   - A white or black window with text will appear

2. **Check Node.js version:**
   - Type this command exactly and press Enter:
   ```
   bash
     node --version
   ```
   - **What to expect:** If installed, you'll see something like `v18.17.0` or `v20.10.0`
   - **What you need:** Version 18.0.0 or higher (the number after the `v`)

3. **Check npm version:**
   - Type this command and press Enter:
   ```
   bash
     npm --version
   ```
   - **What to expect:** You'll see something like `9.6.7` or `10.2.3`

On Windows:

1. **Open Command Prompt:**
   - Click the Start menu (Windows icon)
   - Type "cmd" or "Command Prompt"
   - Click "Command Prompt" from the search results
   - A black window with text will appear

2. **Check Node.js version:**
   - Type this command exactly and press Enter:
   ```
   bash
     node --version
   ```
   - **What to expect:** If installed, you'll see something like `v18.17.0` or `v20.10.0`
   - **What you need:** Version 18.0.0 or higher

3. **Check npm version:**
   - Type this command and press Enter:
   ```
   bash
     npm --version
   ```
   - **What to expect:** You'll see something like `9.6.7` or `10.2.3`

📥 Installing Node.js and npm (If You Don't Have Them)

If the commands above showed errors like "command not found" or the version is below 18.0.0, you need to install or update Node.js.

For Mac:

1. **Go to the official Node.js website:**
   - Visit: https://nodejs.org/ (https://nodejs.org/)

2. **Download the installer:**
   - Click the big green button that says **"LTS"** (Long Term Support)
   - This is the stable, recommended version
   - The download will start automatically (a `.pkg` file)

3. **Install Node.js:**
   - Find the downloaded file (usually in Downloads folder)
   - Double-click the `.pkg` file
   - Follow the installation wizard:
     - Click "Continue" through the prompts
     - Click "Agree" to the license
     - Click "Install"
     - Enter your Mac password when prompted
     - Click "Close" when it says installation was successful

4. **Verify installation:**
   - Close and reopen Terminal (this is important!)
   - Run `node --version` again
   - You should now see a version number like `v20.10.0`

Installation time: About 5 minutes

For Windows:

1. **Go to the official Node.js website:**
   - Visit: https://nodejs.org/ (https://nodejs.org/)

2. **Download the installer:**
   - Click the big green button that says **“LTS”** (Long Term Support)
   - This is the stable, recommended version
   - The download will start automatically (a `.msi` file)

3. **Install Node.js:**
   - Find the downloaded file (usually in Downloads folder)
   - Double-click the `.msi` file
   - Follow the installation wizard:
     - Click “Next”
     - Accept the license agreement and click “Next”
     - Keep the default installation location and click “Next”
     - Keep all default features selected and click “Next”
     - Click “Install”
     - Click “Yes” if Windows asks for permission
     - Click “Finish” when complete

4. **Verify installation:**
   - Close and reopen Command Prompt (this is important!)
   - Run `node --version` again
   - You should now see a version number like `v20.10.0`

Installation time: About 5 minutes

🚀 2. Step-by-Step Running Instructions

Now that you have Node.js and npm installed, let’s run the prototype!

Step 1: Open Your Terminal/Command Prompt

- **Mac:** Press `Command (⌘) + Space`, type “Terminal”, press Enter
- **Windows:** Click Start menu, type “cmd”, press Enter

Step 2: Navigate to the Project Folder

You need to tell your computer where the project files are located.

Copy and paste this command into Terminal/Command Prompt, then press Enter:

```
cd /home/ubuntu/luciteria_collector_cabinet
```

> **Wait, my path is different!**
>
> If you're on Mac or Windows and the project is in a different location (like your Desktop or Documents folder), you'll need to adjust the path. For example:
> - **Mac:** `cd ~/Desktop/luciteria_collector_cabinet`
> - **Windows:** `cd C:\Users\YourName\Desktop\luciteria_collector_cabinet`

What this does: `cd` means "change directory" — it's like opening a folder. We're opening the folder that contains all the prototype files.

What to expect: The terminal prompt will change to show you're now in the project folder. You'll see something like:

```
/home/ubuntu/luciteria_collector_cabinet$
```

Step 3: Install All the Helper Tools (First Time Only)

The prototype needs many helper tools to work. We'll download them all in one command.

Copy and paste this command and press Enter:

```
npm install
```

What this does: This downloads all the "helper libraries" the prototype needs. Think of it like downloading all the apps a new phone needs to work properly.

What to expect:
- You'll see lots of text scrolling by — this is normal!
- You might see some warnings in yellow or orange — don't worry, these are usually safe to ignore
- The process takes 1-3 minutes depending on your internet speed
- When it's done, you'll see text like:
`added 456 packages, and audited 457 packages in 2m`

How long this takes: 1-3 minutes (you only need to do this once)

> **Note:** If you ever move the project folder or update files, you might need to run this command again.

Step 4: Start the Prototype

Now we'll actually start the server that runs the prototype!

Copy and paste this command and press Enter:

```
npm run dev
```

What this does: This starts a mini web server on your computer. The server "hosts" the prototype so you can view it in your web browser.

What to expect:
- You'll see text that says something like:
```

REMIX App Server started

> Local: http://localhost:3000
> Ready in 1.2s
```
- The terminal will "hang" — meaning it won't let you type new commands. This is normal! The server is running.
- **DO NOT CLOSE THIS WINDOW** while you're testing the prototype

How long this takes: 5-15 seconds

Visual Guide: What You Should See

In Terminal/Command Prompt after running `npm run dev`:

```
> luciteria-collector-cabinet@1.0.0 dev
> remix vite:dev

  ➜  Local:   http://localhost:3000/
  ➜  Network: use --host to expose
  ➜  press h to show help

  REMIX vite server ready in 1243 ms
```

This means it's working! ✅

🌐 3. Accessing the Prototype

Opening the Prototype in Your Browser

1. **Open your web browser** (Chrome, Safari, Firefox, Edge — any will work)
2. **Go to this URL:**
   `http://localhost:3000`

Type this EXACTLY into the address bar and press Enter.

1. **What is localhost?**
   - `localhost` is a special web address that means "this computer"
   - `3000` is the "port number" — think of it like an apartment number
   - Together, they point to the mini web server running on your computer

What You'll See When It Loads

When you first visit `http://localhost:3000`, you'll see a simple welcome page or be redirected to the main app.

Expected loading time: Instant to 2 seconds

🗺 All Pages You Can Visit

Here's every URL you can test. Just type these into your browser's address bar:

🏠 Main Entry Points

URL	What It Shows
`http://localhost:3000`	Welcome page / Landing
`http://localhost:3000/app`	Main app dashboard

👤 Customer Pages (Collector View)

URL	What It Shows	What You'll See
`http://localhost:3000/app/cabinet`	**Dashboard**	Overview of your collection progress, recent shipments, quick stats
`http://localhost:3000/app/cabinet/collection`	**Full Collection**	Grid of all 20 elements showing owned, missing, and wishlisted items
`http://localhost:3000/app/cabinet/missing`	**Missing Items**	Browse elements you don't own yet, with filters and details
`http://localhost:3000/app/cabinet/wishlist`	**Wishlist**	Manage your priority wishlist that determines what you get next
`http://localhost:3000/app/cabinet/subscription`	**Subscription**	Your plan details, shipment history, pause/skip options
`http://localhost:3000/app/cabinet/preferences`	**Preferences**	Customize how you handle duplicates and collection preferences

🔧 Admin Pages (Operations View)

URL	What It Shows	What You'll See
`http://localhost:3000/app/admin/operations`	**Operations Dashboard**	Assignment queue, next shipments, exception handling, assignment engine
`http://localhost:3000/app/admin/customers`	**Customer List**	All customers with collection status and subscription tier
`http://localhost:3000/app/admin/customer/cust_001`	**Customer Profile**	Detailed view of one customer's collection, wishlist, shipments

---

🎭 Testing Different Customer Personas

The prototype has 4 different test customers so you can see how the app looks for different collection states.

To switch between customers, add `?customer=cust_XXX` to any customer page URL:

Test Customer 1: Marcus Chen "The Completionist"

```
http://localhost:3000/app/cabinet?customer=cust_001
```

- **Collection state:** 16 out of 20 elements owned (80% complete)
- **Scenario:** Active collector close to completion
- **Wishlist:** 4 specific missing items
- **Best for testing:** Wishlist prioritization, near-complete collection view

Test Customer 2: Sarah Kovacs "The Curator"

```
http://localhost:3000/app/cabinet?customer=cust_002
```

- **Collection state:** 8 out of 20 elements owned (40% complete)
- **Scenario:** Selective collector with curated tastes
- **Best for testing:** Mid-progress collection, preference settings

Test Customer 3: David Nakamura "The Newcomer"

```
http://localhost:3000/app/cabinet?customer=cust_003
```

- **Collection state:** 0 out of 20 elements owned (brand new subscriber)
- **Scenario:** Just starting their collection journey
- **Best for testing:** Empty collection state, first shipment assignment

Test Customer 4: Elena Ross “The Elite Completionist”

```
http://localhost:3000/app/cabinet?customer=cust_004
```

- **Collection state:** 20 out of 20 elements owned (100% complete!)
- **Scenario:** Completed the entire collection
- **Best for testing:** Edge case — what happens when collection is complete

How to use these:
1. Copy any URL above
2. Paste into your browser address bar
3. The page will reload showing that customer’s data
4. You can navigate between pages and the customer ID will persist

---

📖 4. Navigation Guide — All Pages Explained

🎨 Customer Experience Pages

Dashboard (`/app/cabinet`)

What it is: The home page collectors see when they log in.

What’s on it:
- **Progress Ring:** Circular chart showing % of collection completed
- **Collection Stats:** Numbers like “16 owned, 4 missing”
- **Recent Shipments:** Last 3 items received with dates
- **Next Shipment:** Preview of what’s coming next
- **Quick Actions:** Buttons to view wishlist, preferences, etc.

What’s clickable:
- Navigation links to other pages
- Element cards (may link to detail views)
- “View Full Collection” button

Collection Grid (`/app/cabinet/collection`)

What it is: Visual grid showing all 20 elements.

What’s on it:
- **Grid of Element Cards:** Each showing:
- Element symbol (Au, Ag, Cu, etc.)
- Element name (Gold, Silver, Copper, etc.)
- Status badge (Owned, Missing, or Wishlisted)
- Visual state (owned items are highlighted)

What’s clickable:
- Each element card
- Filter buttons (show all / owned / missing / wishlisted)
- Heart icon to add/remove from wishlist

What to look for:
- How owned vs. missing elements look different
- Wishlist heart icons
- Rarity indicators on missing items

Missing Items (`/app/cabinet/missing`)

What it is: Browse elements you don't own yet.

What's on it:
- **List of unowned elements** with:
- Element details (symbol, name, number)
- Availability status (available / temporarily unavailable)
- Rarity indicator (common, rare, ultra-rare)
- "Add to Wishlist" button

What's clickable:
- Add to Wishlist button
- Filter dropdowns (by availability, rarity)
- Sort options

What to test:
- Adding items to wishlist
- Filtering by rarity
- Seeing availability status

Wishlist (`/app/cabinet/wishlist`)

What it is: Manage your priority list that determines what you receive next.

What's on it:
- **Ordered list of wishlisted elements** showing:
- Priority rank (1, 2, 3, etc.)
- Element details
- Drag handles to reorder
- Remove button

What's clickable:
- Drag handles to reorder priorities
- Remove from wishlist button
- "Save Changes" button

What to test:
- Reordering wishlist items
- Removing items
- Understanding how priority affects assignments

Subscription (`/app/cabinet/subscription`)

What it is: Manage your subscription plan.

What's on it:
- **Current Plan Details:**
- Tier name (Premier, Premium)
- Price
- Shipment frequency
- Next shipment date
- **Shipment History:** Past deliveries with dates
- **Actions:** Skip next shipment, pause subscription, cancel

What's clickable:
- Skip shipment button
- Pause/resume button
- Cancel subscription link
- Shipment history items

What to test:
- Viewing plan details
- Shipment history
- Understanding skip/pause options

Preferences (`/app/cabinet/preferences`)

What it is: Customize how your collection works.

What's on it:
- **Duplicate Handling Mode:**
- Strict (never send duplicates)
- Flexible (allow duplicates if wishlist empty)
- **Category Preferences:**
- Preferred element types
- Format preferences
- **Save Settings** button

What's clickable:
- Radio buttons for duplicate mode
- Category checkboxes
- Save button

What to test:
- Changing duplicate handling mode
- Selecting preferences
- Saving changes

🔧 Admin Experience Pages

Operations Dashboard (`/app/admin/operations`)

What it is: The control center for managing all customer shipments.

What's on it:
- **Assignment Queue:** Next shipments to process

- **Pending Assignments:** Customers waiting for assignments
- **Assignment Engine Section:**
- “Run Assignment Engine” button
- Live algorithm status
- Assignment results
- **Exception Handling:** Edge cases and errors
- **Stats Dashboard:** Total customers, active subscriptions, shipments this month

What’s clickable:
- “Run Assignment Engine” button (demonstrates the algorithm)
- Customer links
- Queue items

What to test:
- Running the assignment engine
- Viewing assignment results
- Understanding how the algorithm prevents duplicates

Customers List (`/app/admin/customers`)

What it is: Table of all customers.

What’s on it:
- **Customer Table** with columns:
- Customer name
- Collection progress (e.g., “16/20 owned”)
- Subscription tier
- Status (active, paused, cancelled)
- Next shipment date

What’s clickable:
- Customer names (links to individual profile)
- Column headers (for sorting)
- Filter buttons

What to test:
- Sorting by different columns
- Clicking to view individual customer

Individual Customer Profile (`/app/admin/customer/:id`)

What it is: Deep dive into one customer’s account.

What’s on it:
- **Customer Info:** Name, email, tier, join date
- **Collection Overview:** Progress, owned items, wishlist
- **Shipment History:** All past deliveries
- **Next Assignment:** What they’ll receive next and why
- **Assignment Algorithm Explanation:** Shows logic used
- **Actions:** Override assignment, pause subscription

What's clickable:
- Navigation back to customer list
- Owned element badges
- Shipment details
- Override assignment button

What to test:
- Understanding why specific items were assigned
- Viewing full collection state
- Assignment logic explanation

Try these customer IDs:
- `/app/admin/customer/cust_001` — Marcus Chen
- `/app/admin/customer/cust_002` — Sarah Kovacs
- `/app/admin/customer/cust_003` — David Nakamura
- `/app/admin/customer/cust_004` — Elena Ross

🆘 5. Common Issues & Troubleshooting

❌ Problem: "Port 3000 is already in use"

What this means: Another program is already using port 3000, so our prototype can't start there.

Error message you'll see:

```
Error: listen EADDRINUSE: address already in use :::3000
```

How to fix it:

Option 1: Find and Stop the Other Program (Mac)

1. **Find what's using port 3000:**
   `bash`
   `lsof -i :3000`
2. **Stop that process:**
   - Look for the number under the "PID" column
   - Run this command (replace `1234` with the actual PID):
   `bash`
   `kill -9 1234`
3. **Try running the prototype again:**
   `bash`
   `npm run dev`

Option 1: Find and Stop the Other Program (Windows)

1. **Find what's using port 3000:**
   `bash`
   `netstat -ano | findstr :3000`
2. **Stop that process:**
   - Look for the number in the last column (PID)

- Run this command (replace `1234` with the actual PID):
`bash`
`taskkill /PID 1234 /F`

3. **Try running the prototype again:**
`bash`
`npm run dev`

Option 2: Use a Different Port

You can run the prototype on a different port number:

```
PORT=3001 npm run dev
```

Then visit `http://localhost:3001` instead.

❌ Problem: “Module not found” or “Cannot find module”

What this means: The helper tools didn’t install correctly.

Error message you’ll see:

```
Error: Cannot find module 'react'
```

How to fix it:

1. **Make sure you’re in the project folder:**
`bash`
`cd /home/ubuntu/luciteria_collector_cabinet`

2. **Delete the old installation and reinstall:**
`bash`
`rm -rf node_modules package-lock.json`
`npm install`

3. **Try running the prototype again:**
`bash`
`npm run dev`

How long this takes: 2-4 minutes to reinstall everything

❌ Problem: “Database error” or “Prisma error”

What this means: The database setup wasn’t completed or got corrupted.

Error message you’ll see:

```
PrismaClientInitializationError: Can't reach database server
```

How to fix it:

This prototype works with or without a database. By default, it uses mock data (fake data stored in memory), so you shouldn't see database errors.

If you do:

1. **Reset everything:**
   `bash`
   `npm run setup`

This will:
- Install all dependencies
- Set up the database
- Fill it with test data

1. **Try running the prototype again:**
   `bash`
   `npm run dev`

How long this takes: 2-3 minutes

❌ Problem: Browser Shows Blank White Page

What this means: The prototype started but the page isn't loading correctly.

How to fix it:

1. **Check if the server is actually running:**
   - Look at your Terminal/Command Prompt window
   - You should see "REMIX vite server ready"
   - If not, try running `npm run dev` again

2. **Hard refresh the browser:**
   - **Mac:** Press `Command (⌘) + Shift + R`
   - **Windows:** Press `Ctrl + Shift + R`

3. **Check the browser console for errors:**
   - **Mac:** Press `Command (⌘) + Option + J`
   - **Windows:** Press `Ctrl + Shift + J`
   - Look for red error messages
   - Copy any error messages and search for them online

4. **Try a different browser:**
   - If you're using Safari, try Chrome
   - If you're using Edge, try Firefox

5. **Check the exact URL:**
   - Make sure you're visiting `http://localhost:3000` exactly
   - Make sure it's `http` not `https`

❌ Problem: "node: command not found"

What this means: Node.js isn't installed or isn't in your system's PATH.

How to fix it:

1. **Go back to the Prerequisites section** and install Node.js
2. **After installing, close and reopen Terminal/Command Prompt**
   - This is crucial! The terminal needs to restart to recognize the new software
3. **Verify Node.js is installed:**
   ```
   bash
      node --version
   ```

❌ Problem: Changes to Code Aren't Showing Up

What this means: The development server needs to be restarted.

How to fix it:

1. **Stop the server:**
   - Go to the Terminal/Command Prompt window where it's running
   - Press `Ctrl + C` (on both Mac and Windows)
   - You'll see text like `^C` and get your command prompt back
2. **Restart the server:**
   ```
   bash
      npm run dev
   ```
3. **Hard refresh your browser:**
   - **Mac:** `Command (⌘) + Shift + R`
   - **Windows:** `Ctrl + Shift + R`

🛑 How to Stop the Server When You're Done

When you're finished testing the prototype, you should stop the server:

1. **Go to the Terminal/Command Prompt window** where the server is running
2. **Press:**
   `Ctrl + C`
   (This works on both Mac and Windows)
3. **What to expect:**
   - The text will stop scrolling
   - You'll see `^C`
   - Your command prompt will return (you can type again)
   - The server is now stopped
4. **Verify it stopped:**
   - Try visiting `http://localhost:3000` in your browser
   - It should show an error like "This site can't be reached"
5. **You can now close the Terminal/Command Prompt window**

💡 Other Tips

The Terminal Window Looks Frozen

- **This is normal!** When the server is running, the terminal doesn't let you type new commands
- You'll see logs appear when you interact with the prototype in your browser
- To stop the server and get your command prompt back, press `Ctrl + C`

Nothing Happens When I Click Buttons

- Some buttons in the prototype are just for visual demonstration
- See the "What You're Looking At" section below to understand what's interactive vs. what's just mockup

The Page Looks Broken or Unstyled

- Wait a few more seconds — styles take a moment to load
- Try hard refreshing: `Cmd+Shift+R` (Mac) or `Ctrl+Shift+R` (Windows)
- Check your internet connection (some fonts/resources load from online)

💡 6. What You're Looking At

This is a Prototype, Not a Live Product

This is an **interactive prototype** — a working demo built to show what the Collector Cabinet would look like and how it would work. It's not connected to real Shopify stores or real customer data.

What's Real vs. What's Mock Data

✅ Real (Actually Works):

- **Navigation** — All pages and links work
- **Visual Design** — This is the actual UI/UX design
- **Page Layouts** — Exactly how the real app would look
- **Assignment Algorithm** — The logic for preventing duplicates is real and working
- **Filtering/Sorting** — Interactive elements function correctly

🎭 Mock (Fake for Demo):

- **Customer Data** — The 4 test customers are fictional
- **Element Inventory** — Pre-set test data (only 20 elements)
- **Shipment History** — Fake past orders
- **Date/Times** — Simulated timestamps
- **Actions** — Buttons work visually but don't persist data

How the Assignment Logic Works in the Demo

The prototype includes a working version of the **duplicate-prevention algorithm**. Here's how it works:

When You Click "Run Assignment Engine" in Admin Operations:

1. **The system checks each customer's collection:**
   - What elements do they already own?

- What elements are on their wishlist?
- What's their duplicate handling mode?

2. **For each customer, it assigns the next item by:**
   - ✅ First priority: Highest item on wishlist they don't own
   - ✅ Second priority: Any available element they don't own
   - ⚠️ Edge case: If they own everything and allow duplicates, assign a duplicate
   - ❌ If they own everything and disallow duplicates, flag as exception

3. **Results are displayed:**
   - You'll see which element was assigned to each customer
   - You'll see the reasoning (e.g., "Top wishlist item: Gold")
   - Duplicates are prevented automatically

You Can Test This:

1. Go to `/app/admin/operations`
2. Click "Run Assignment Engine"
3. See the results show which elements would ship to which customers
4. Check that no duplicates are assigned (unless customer preferences allow it)

What Would Be Different in Production

If this were a real, live Shopify app, here's what would change:

Real Data Sources:

- Customer data from Shopify customer database
- Real product inventory from Shopify products
- Actual subscription billing through Shopify checkout
- Real order history from Shopify orders

Real Integrations:

- Shopify Admin API for product management
- Shopify Storefront API for customer views
- Subscription app integration (Recharge, Seal, etc.)
- Real payment processing

Real Workflows:

- Automatic nightly assignment runs (not manual button)
- Email notifications for shipments
- Real package tracking
- Customer support integration

Data Persistence:

- Changes you make would save to a real database
- Wishlists would persist between sessions
- Collection progress would update as real orders ship

Security & Authentication:

- Real Shopify OAuth login

- Customer-specific accounts
- Admin role permissions
- Secure data encryption

Scale:

- Hundreds or thousands of real customers
- Full catalog of elements (100+ products)
- Real inventory management
- Production-grade servers

Interactive vs. Visual-Only Elements

🖱 Fully Interactive (Click and They Work):

- ✅ Navigation menu
- ✅ Page links
- ✅ "Run Assignment Engine" button
- ✅ Customer switcher (`?customer=cust_001`)
- ✅ Sorting/filtering on some pages
- ✅ Add/remove wishlist buttons (in demo context)

👁 Visual Only (Just for Design Reference):

- 📋 Form submissions (buttons exist but don't save to database)
- 📋 Pause/cancel subscription (shows UI but doesn't persist)
- 📋 Element detail modals (may not open)
- 📋 Some action buttons (they show feedback but don't change data permanently)

Rule of thumb: If you click something and the page changes or shows a message, it's working in prototype mode. But remember, refreshing the page will reset any changes since it's using temporary mock data.

Testing Tips

See the Full Experience:

1. Start with customer view: `http://localhost:3000/app/cabinet?customer=cust_003` (new customer)
2. Browse missing items
3. Add items to wishlist
4. Check the collection grid
5. Switch to admin view: `http://localhost:3000/app/admin/operations`
6. Run the assignment engine
7. See what the new customer gets assigned

Test Edge Cases:

- View the completed collector: `?customer=cust_004` — what do they see?
- View the operations dashboard when someone has no wishlist
- Try all 4 customer personas to see different states

Design Review:

- Note the dark theme and brand colors
- Check how element status is visually indicated
- Review the language and tone (“The Hunt Continues” vs. “Missing Items”)
- See how rarity and scarcity are presented as exciting

🎉 You’re All Set!

You should now be able to:
- ✅ Install Node.js and npm
- ✅ Run the prototype locally
- ✅ Navigate all customer and admin pages
- ✅ Test different customer personas
- ✅ Understand what’s real vs. mock
- ✅ Troubleshoot common issues
- ✅ Stop the server when done

Quick Reference Card

Start the prototype:

```
cd /home/ubuntu/luciteria_collector_cabinet
npm run dev
```

Visit in browser:

```
http://localhost:3000
```

Stop the server:

```
Ctrl + C (in Terminal/Command Prompt)
```

Switch customers:

```
Add ?customer=cust_001 to any /app/cabinet URL
```

❓ Still Have Questions?

If you run into issues not covered here:

1. **Check the error message carefully** — copy the exact text
2. **Search the error online** — Stack Overflow usually has answers
3. **Check the Terminal/Command Prompt** for more detailed errors
4. **Try the troubleshooting steps** in Section 5 above

5. **Reach out to the developer** with:
   - The exact error message
   - What command you ran
   - Screenshot of Terminal/Command Prompt

Happy testing! 🚀

Last updated: May 21, 2026